

Trí tuệ nhân tạo

chương 2

GIẢI QUYẾT VẤN ĐỀ BẰNG TÌM KIẾM

Tổng quan

- Vấn đề là gì?
- Giải quyết vấn đề là làm gì?

Vấn đề?

- Vấn đề là một bài toán nào đó cần giải quyết: một câu đố, một trò chơi, chứng minh một định lý,...
- Giải quyết vấn đề là quá trình xuất phát từ **hình trạng đầu**, tìm kiếm trong không gian bài toán để tìm ra **dãy toán tử** hay **dãy hành động** cho phép dẫn tới **đích**.

3/2018

3

Bài toán đong nước

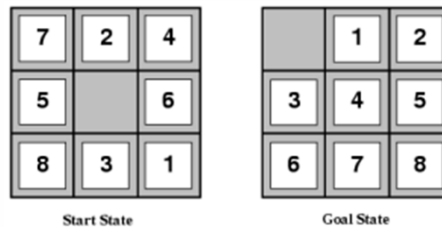
- Cho 2 bình A(m lít), B(n lít). Làm cách nào để đong được k lít ($k \leq \min(m,n)$) chỉ bằng 2 bình A, B và 1 bình trung gian C.
- Các thao tác:
 $C \rightarrow A$; $C \rightarrow B$; $A \rightarrow B$; $B \rightarrow A$;
- Điều kiện: không tràn, đổ hết

3/2018

4

Bài toán trò chơi $n^2 - 1$ số

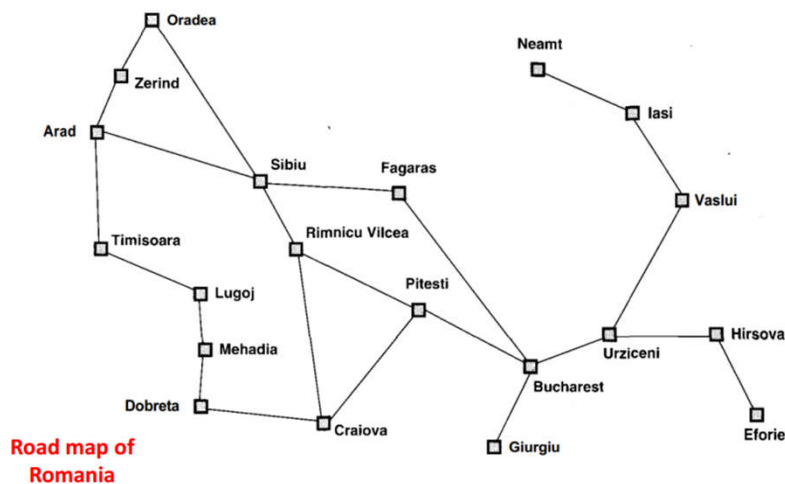
- Trong bảng ô vuông n hàng, n cột, mỗi ô chứa 1 số nằm trong phạm vi từ $1 \rightarrow n^2 - 1$ sao cho không có 2 ô có cùng giá trị. Còn đúng 1 ô bị trống.
- Xuất phát từ một sắp xếp nào đó các số trong bảng, hãy dịch chuyển ô trống sang phải, sang trái, lên trên hoặc xuống dưới (nếu có thể được) để đưa bảng ban đầu về bảng qui ước trước



3/2018

5

Bài toán tìm đường đi

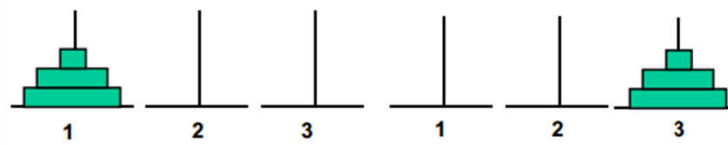


3/2018

6

Bài toán tháp Hà Nội

- Cho 3 cọc 1, 2, 3. Ở cọc 1 ban đầu có n đĩa, sắp theo thứ tự nhỏ dần từ dưới lên trên.
- Hãy tìm cách chuyển n đĩa đó sang cọc 3 sao cho:
 - Mỗi lần chỉ chuyển 1 đĩa
 - Ở mỗi cọc không cho phép đĩa to nằm trên đĩa nhỏ



Bài toán tháp Hà Nội với $n = 3$

3/2018

7

Một số phương pháp biểu diễn vấn đề

- Biểu diễn nhờ KGTT
 - Mỗi hình trạng của bài toán tương ứng với 1 trạng thái (state)
 - Mỗi phép biến đổi từ hình trạng này sang hình trạng khác tương ứng với các toán tử (operator)
- Qui bài toán về bài toán con
 - Phân chia bài toán thành các bài toán con, các bài toán con lại được phân rã tiếp cho đến khi gặp được các bài toán sơ cấp cho phép xác định lời giải của bài toán ban đầu trên cơ sở lời giải của các bài toán con

3/2018

8

Một số phương pháp biểu diễn vấn đề

- Sử dụng logic hình thức
 - logic mệnh đề
 - logic vị từ cấp 1

3/2018

9

Giải quyết vấn đề bằng tìm kiếm

- Tìm kiếm là vấn đề quan trọng trong TTNT:
 - Tìm chuỗi hành động nhằm tối đa kết quả trong tương lai (lập kế hoạch)
 - Tìm kiếm trong CSTT để tìm chỗi các hành động có thể thực hiện trong tương lai (suy luận logic, xác suất)
 - Tìm các mô hình phù hợp với các quan sát (trong học máy)
- Tìm kiếm là một trong những thành công của các nghiên cứu về TTNT giai đoạn đầu

3/2018

10

Chương 2: Giải quyết vấn đề bằng tìm kiếm

- I. Biểu diễn vấn đề trong không gian tìm kiếm
- II. Các chiến lược tìm kiếm mù
- III. Chiến lược tìm kiếm kinh nghiệm (heuristic)
- IV. Chiến lược tìm kiếm tối ưu
- V. Tìm kiếm có đối thủ

I. Biểu diễn vấn đề trong không gian trạng thái

- 1. Không gian trạng thái
- 2. Biểu diễn vấn đề trong không gian tìm kiếm

1. Không gian trạng thái (KGTT)

- Khi giải quyết bài toán bằng phương pháp tìm kiếm, trước hết ta phải xác định *không gian tìm kiếm* - bao gồm tất cả các đối tượng trên đó ta thực hiện việc tìm kiếm.
- Một phương pháp biểu diễn vấn đề phù hợp là sử dụng các khái niệm *trạng thái* (state) và *toán tử* (operator).
- Phương pháp giải quyết vấn đề dựa trên khái niệm trạng thái và toán tử được gọi là cách tiếp cận giải quyết vấn đề nhờ không gian trạng thái (KGTT).

3/2018

13

Định nghĩa KGTT

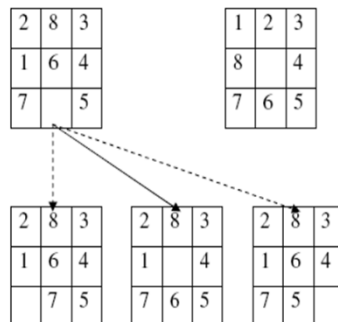
- KGTT là một bộ bốn (S, s_0, F, G) , trong đó:
 - S là tập các trạng thái
 - s_0 là trạng thái bắt đầu
 - F là tập các toán tử/các phép biến đổi.
 - G là tập trạng thái đích.

3/2018

14

Các thành phần của KGTT

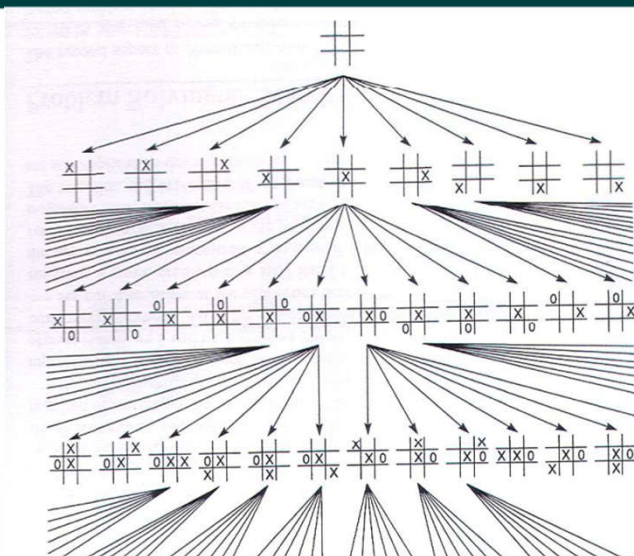
- Sự sắp xếp các số tại mỗi thời điểm là một TT.
- Hình bên trái là trạng thái ban đầu.
- Hình bên phải là trạng thái kết thúc hay trạng thái đích (goal).
- Trạng thái đích của một bài toán có thể nhiều hơn một (vd?)
- Toán tử là một phép biến đổi hợp lệ chuyển từ trạng thái này sang trạng thái khác.
- Bằng các toán tử, từ trạng thái ban đầu, tiếp tục phát triển, cuối cùng thu được một không gian trạng thái (KGTT).



3/2018

15

Ví dụ trò chơi cờ ca-rô (Tic-Tac-Toe)

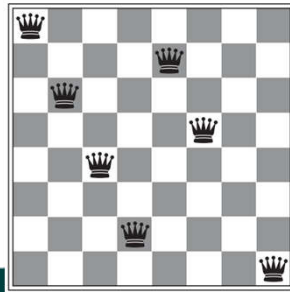


3/2018

16

Bài toán 8 quân hậu

- Mục tiêu của bài toán 8 quân hậu là đặt tám quân hậu trên một bàn cờ sao cho không quân hậu nào tấn công quân hậu nào.
 - Quân hậu tấn công bất kỳ quân nào trong cùng một hàng, cùng cột hoặc cùng đường chéo.
 - Ví dụ: Hình dưới đây cho thấy một giải pháp đã thử không thành công: quân hậu ở cột ngoài cùng bên phải bị quân hậu ở trên cùng bên trái tấn công.



3/2018

17

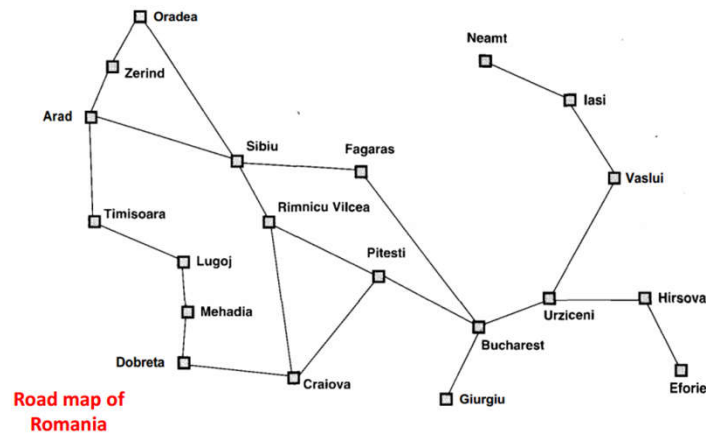
■ Không gian trạng thái:

- Trạng thái: Bất kỳ sự sắp xếp nào từ 0 đến 8 quân hậu trên bàn cờ đều là một trạng thái
- Trạng thái ban đầu: Không có quân hậu nào trên bàn cờ.
- Hành động: Thêm quân hậu vào ô trống bất kỳ
- Trạng thái đích: 8 quân hậu trên bàn cờ, không quân nào bị tấn công.

3/2018

18

■ Bài toán đi tham quan các thành phố?

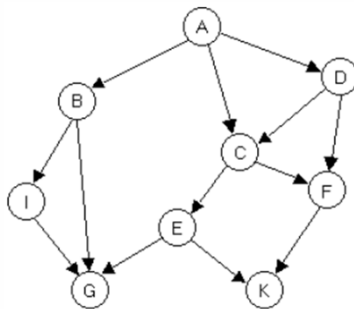


3/2018

19

Biểu diễn KGTT

- KGTT có thể biểu diễn bằng *đồ thị có hướng*: mỗi đỉnh là một trạng thái, mỗi cung là một toán tử.
- Bài toán có *nghiệm* nếu ta tìm được đường đi từ trạng thái bắt đầu đến một trong các trạng thái đích.



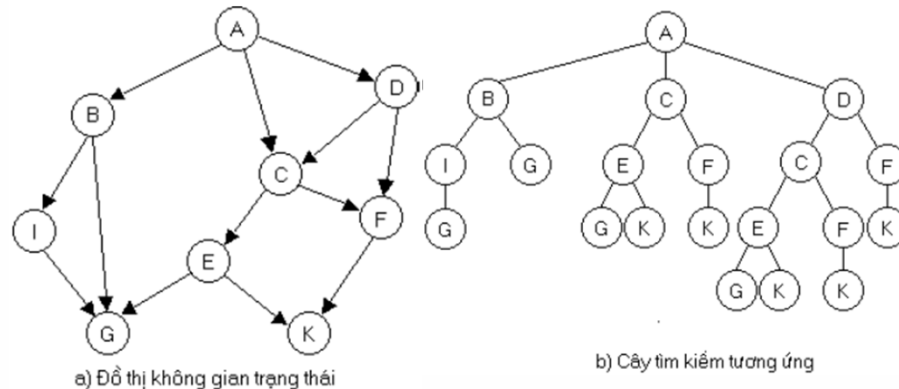
a) Đồ thị không gian trạng thái

3/2018

20

Biểu diễn bằng cây

- Trong đồ thị của KGTT có thể xuất hiện chu trình gây khó khăn cho việc tìm kiếm, hạn chế các toán tử trong F có thể đưa đồ thị trở thành cây, một cấu trúc dễ tìm kiếm hơn.



3/2018

21

Giải quyết vấn đề bằng tìm kiếm

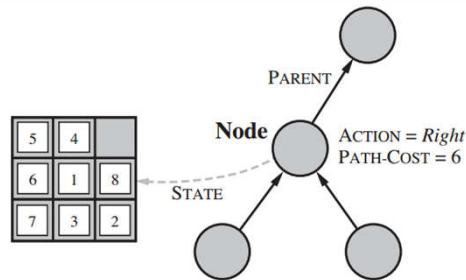
- Tập trạng thái
- Bước chuyển/ toán tử
- Chi phí bước chuyển (path cost)
- Hình trạng/trạng thái đích
- Hình trạng/trạng thái xuất phát
- Lối giải = Các bước chuyển từ trạng thái xuất phát đến trạng thái đích
- Tìm lối giải = Tìm đường đi
 - Tìm càng nhanh thì càng thông minh?

3/2018

22

Thông tin mỗi nút?

- Nội dung trạng thái mà nút hiện hành đang biểu diễn.
- Nút cha đã sinh ra nó.
- Toán tử đã được sử dụng để sinh ra nút hiện hành.
- Độ sâu của nút.
- Giá trị đường đi từ nút gốc đến nút hiện hành.



3/2018

23

Chiến lược tìm kiếm?

- Khi tìm kiếm lời giải, từ một trạng thái nào đó chưa phải là trạng thái đích, ta dựa theo toán tử sinh ra tập các trạng thái mới - mở rộng.
- Để được lời giải, ta phải liên tục *chọn trạng thái mới, mở rộng, kiểm tra* cho đến khi tìm được trạng thái đích hoặc không mở rộng được KGTT.
- Tập các trạng thái được mở rộng sẽ có nhiều phần tử, việc *chọn* trạng thái nào để tiếp tục mở rộng được gọi là chiến lược tìm kiếm.

3/2018

24

Đánh giá một chiến lược?

- Tính đầy đủ: chiến lược phải đảm bảo tìm được lời giải, nếu có.
- Độ phức tạp thời gian: mất thời gian bao lâu để tìm được lời giải?
- Độ phức tạp không gian: tốn bao nhiêu đơn vị bộ nhớ để tìm được lời giải?
- Tính tối ưu: tốt hơn so với một số chiến lược khác hay không?

3/2018

25

Nội dung

- I. Biểu diễn vấn đề trong không gian tìm kiếm
- II. Các chiến lược tìm kiếm mù
- III. Chiến lược tìm kiếm kinh nghiệm (heuristic)
- IV. Chiến lược tìm kiếm tối ưu
- V. Tìm kiếm có đối thủ

3/2018

26

II. Các chiến lược tìm kiếm mù

3/2018

27

Tìm kiếm mù?

- Trạng thái được chọn để phát triển chỉ đơn thuần dựa theo cấu trúc của KGTT mà không có thông tin hướng dẫn nào khác.
- Nói chung tìm kiếm mù sẽ không hiệu quả.
- Đây là cơ sở để chúng ta cải tiến và thu được những chiến lược hiệu quả hơn.

3/2018

28

II. Các chiến lược tìm kiếm mù

1. Tìm kiếm theo chiều rộng
2. Tìm kiếm theo chiều sâu
3. Tìm kiếm theo độ sâu hạn chế
4. Tìm kiếm sâu dần

3/2018

29

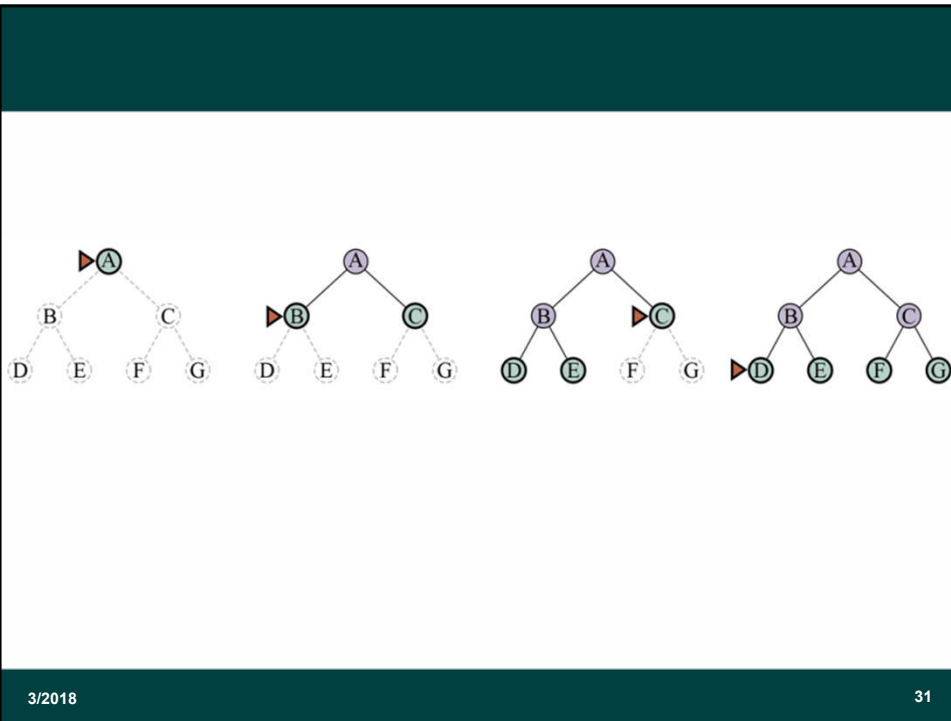
1. Tìm kiếm theo chiều rộng (BFS)

BFS: **B**readth-**f**irst search

- Trạng thái được ưu tiên phát triển là trạng thái được sinh ra trước.
- Dùng danh sách **open** chứa các trạng thái sinh ra đang chờ phát triển
- Danh sách **closed** chứa các trạng thái đã được khảo sát.

3/2018

30



3/2018

31

Thuật toán BFS

```

procedure bfs;
begin
  open:=[start]; closed:=[];
  while open<>[] do
    begin
      loại trạng thái ngoài cùng bên trái của open, gọi nó là u
      if (u là một đích) then thông báo kết quả, thoát
      else begin
        Đưa u vào closed
        Phát sinh các con v của u
        Loại các con đã có trong open+closed
        Đưa các con còn lại vào bên phải open
      end
    end
    Thông báo thất bại
  End

```

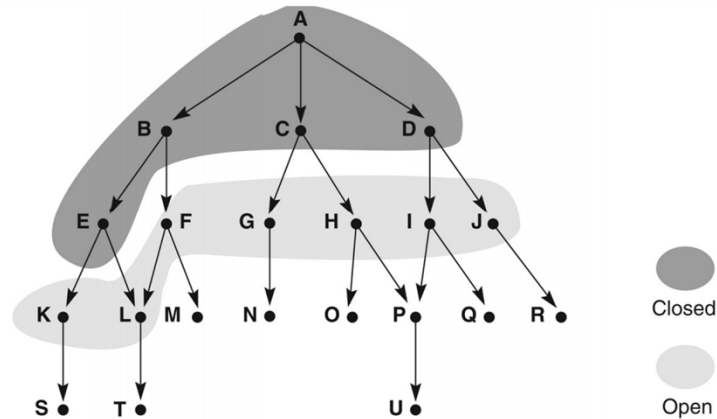
Open: VÀO TRƯỚC – RA TRƯỚC

3/2018

32

BFS - Ví dụ

- VD: Xác định các tập Open và Closed ở từng bước



3/2018

33

Nhận xét

- Các trạng thái con phát sinh nhờ các toán tử hợp lệ.
- Danh sách open bổ sung phần tử bên phải, lấy phần tử bên trái.
- Thuật toán khảo sát tất cả các nút ở độ sâu d sau đó mới đến mức $d+1$ nên chắc chắn tìm được nghiệm.
- Nếu vô nghiệm và KGTT hữu hạn thì thuật toán sẽ dừng và thông báo vô nghiệm.

3/2018

34

Đánh giá

- Giả sử mỗi trạng thái trung bình sinh ra b trạng thái con (kề), b - gọi là **nhân tố nhánh**.
- Giả sử đường đi nghiệm có độ sâu d . Tình trạng xấu nhất phải khảo sát là ?
- Độ phức tạp thời gian là $O(b^d)$, độ phức tạp không gian cũng là $O(b^d)$.

3/2018

35

- Giả sử để phát hiện và kiểm tra 1000 trạng thái cần 1 giây, và lưu giữ 1 trạng thái cần 100 bytes

Độ sâu	Số nút	Thời gian	Bộ nhớ
2	111	0.1 giây	11 kilobytes
4	11,111	11 giây	1 megabyte
6	10^6	18 phút	111 megabytes
8	10^8	31 giờ	11 gigabytes
10	10^{10}	128 ngày	1 terabyte
12	10^{12}	35 năm	111 terabyte
14	10^{14}	3500 năm	11.111 terabyte

Bảng 1. Thời gian, bộ nhớ cần thiết cho tìm kiếm theo bề rộng với $b=10$.

3/2018

36

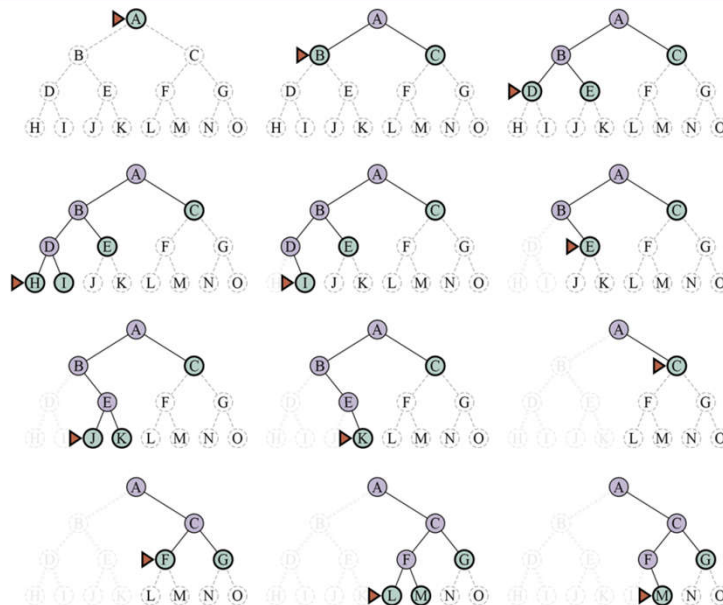
2. Tìm kiếm theo chiều sâu (DFS)

Deep-First Search (DFS)

- Mở rộng nút có độ sâu hơn trước các nút khác đang chờ xử lý.
- Khi nào không mở rộng được nữa thì mới quay lại nút ở độ sâu thấp hơn.

3/2018

37



3/2018

38

Thuật toán DFS

```

procedure dfs;
begin
  open:=[start]; closed:=[];
  while open<>[] do
    begin
      loại trạng thái u ngoài cùng bên trái của open
      if (u là một đích) then thông báo kết quả, thoát
      else begin
        Đưa u vào closed
        Phát sinh các con v của u
        Loại các con đã có trong open+closed
        Đưa các con còn lại vào bên trái open
      end
    end
    Thông báo thất bại
  End

```

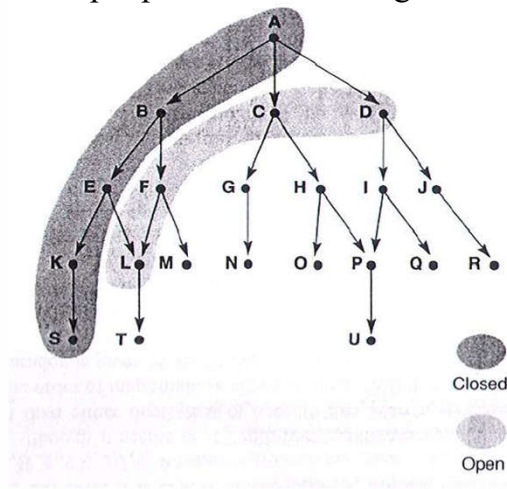
Open: VÀO SAU – RA TRƯỚC

3/2018

39

DFS – ví dụ

- VD: Xác định các tập Open và close ở từng bước



3/2018

40

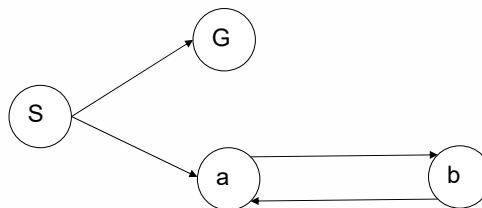
Nhận xét

- Khảo sát nút ở độ sâu d thì DFS lưu trữ $b \cdot d$ nút, trong khi đó BFS phải lưu trữ b^d nút.
- Ở độ sâu $d=12$, DFS chỉ sử dụng 12KB trong khi BFS dùng đến 111TB.
- Độ phức tạp thời gian của DFS vẫn là $O(b^m)$ vì trong trường hợp xấu nhất các nút được khảo sát vẫn như BFS.
- Cơ hội để tìm thấy đích nhanh hơn nếu nó nằm ở phần KGTT bên trái.

3/2018

41

- Ví dụ trường hợp dfs lặp vô hạn $\rightarrow$ không đầy đủ, không tối ưu



3/2018

42

Hạn chế

- Bỏ qua cơ hội tìm thấy ngay trạng thái đích khi trạng thái này nằm gần gốc.
- Nếu KGTT vô hạn có thể không tìm được trạng thái đích.
- Nghiệm nói chung không phải là đường đi ngắn nhất.
- DFS là chiến lược không (hoàn toàn) đầy đủ, không tối ưu. Không nên sử dụng khi KGTT có độ sâu lớn hoặc vô hạn.

3/2018

43

3. Tìm kiếm với độ sâu hạn chế (DFS(d))

- Vẫn như sử dụng chiến lược tìm kiếm theo chiều sâu nhưng giới hạn độ sâu của đường đi nghiệm trên cây.
- Không tiếp tục mở rộng nếu đã đến một độ sâu d cố định nào đó.
- Số d được gọi là bán kính trên KGTT.

Nhận xét:

- Chiến lược này sẽ tìm được nghiệm hay không phụ thuộc vào d .
- Nếu d được chọn thích hợp thì nó tìm được nghiệm, khi đó chiến lược là đầy đủ. Tuy nhiên nó không là chiến lược tối ưu.
- Tương tự như DFS, độ phức tạp thời gian là $O(b^d)$ và độ phức tạp không gian là $O(bd)$.

3/2018

44

Thuật toán

```

procedure DFS(d) ;
begin
  open:=[start]; closed:=[]; depth(start):=0;
  while open<>[] do
    begin
      loại u ngoài cùng bên trái open
      if (u là một đích) then thông báo kết quả, thoát
      else
        begin
          Đưa u vào closed
          If depth(u)<d then begin
            Phát sinh các con v của u
            Loại các con vừa phát sinh đã có trong open+closed
            Gán độ sâu cho các v bằng depth(u)+1
            Đưa các con v còn lại vào bên trái open
          End;
        End;
      End; {while}
    Thông báo thất bại
  End.

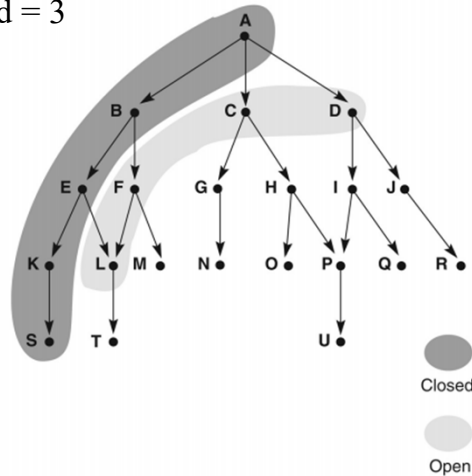
```

3/2018

45

DFS(d) – ví dụ

- VD: Xác định các tập Open và close ở từng bước với độ sâu hạn chế $d = 3$



3/2018

46

DFS(d)

Khó khăn?

- Vấn đề khó khăn là xác định độ sâu hạn chế: d ?
- Hầu hết các bài toán chúng ta không biết trước d bằng bao nhiêu.

→ Tìm kiếm sâu dần.

3/2018

47

4. Tìm kiếm sâu dần (IDS-Iterative-Deepening Search)

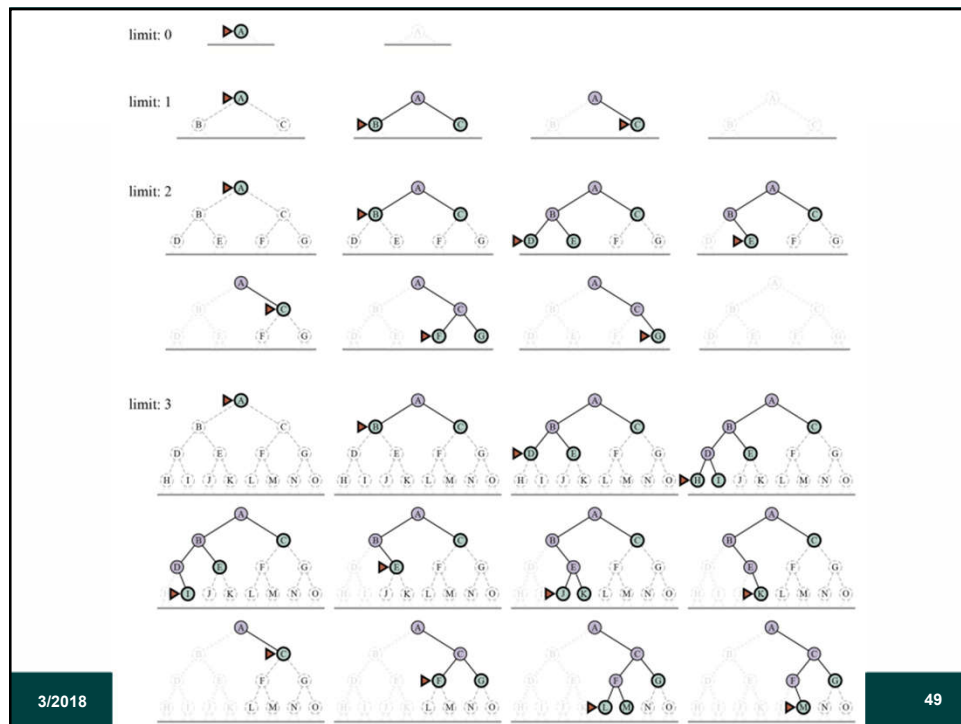
- Từng bước tăng dần độ sâu hạn chế để tìm nghiệm.
- Lần lượt cho d bằng $1, 2, \dots$ và tìm kiếm với độ sâu hạn chế ứng với d .
- Tận dụng được lợi thế của hai chiến lược tìm kiếm rộng và tìm kiếm sâu vì nó tìm hết cây độ sâu d đến $d+1$ giống như tìm kiếm rộng

Nhận xét:

- Chắc chắn tìm được nghiệm nếu có.
- Do sử dụng tìm kiếm sâu trong cây hạn chế nên ít tốn kém bộ nhớ hơn.
- Chiến lược này là đầy đủ và nếu KGTT là cây thì nó là chiến lược tối ưu.

3/2018

48



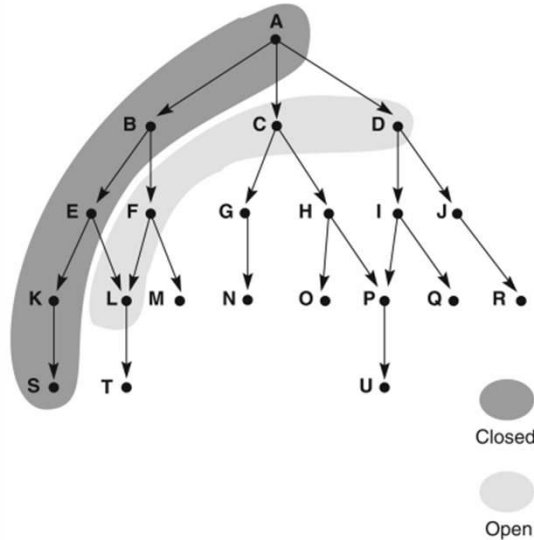
Thuật toán

```

procedure IDS;
begin
  for d:=0 to max do
    begin
      DFS(d) ;
      If thành công then exit
    End;
  End;

```

Tìm kiếm sâu dần – ví dụ



3/2018

51

Đánh giá:

- Thực tế số nút tăng lên nhưng độ phức tạp của thuật toán vẫn là $O(b^d)$, trong khi đó độ phức tạp không gian vẫn là $O(bd)$.
- Vì vậy, tìm kiếm sâu dần được đánh giá là chiến lược tìm kiếm thích hợp hơn khi KGTT lớn và không biết trước được độ sâu của trạng thái đích.

3/2018

52

Các chiến lược tìm kiếm mù

- Trạng thái được chọn để phát triển chỉ đơn thuần dựa theo cấu trúc của KGTT mà không có thông tin hướng dẫn nào khác.
- Thông tin về TT đích không đóng vai trò trong tìm kiếm.
- Kém hiệu quả và không thể áp dụng được trong nhiều trường hợp.
- Sử dụng thông tin của trạng thái kết hợp với nhận xét dựa theo kinh nghiệm để cải tiến là quan điểm chung của các chiến lược *tìm kiếm kinh nghiệm*.